Using the Image Plugin

resize, convert, crop, filter, watermark, placeholder

GD • Imagick • Vips

AlfacodeTeam PhpServicePlatform — `image.processing`

Every number recorded from real engines via `docs/generators/probe_image.php`

Engines on this run: GD, Imagick, Vips

Contents

1 What this guide is **2**

2 Install and choose a driver **2**

2.1 Three engines, one contract . . . 2

2.2 Configuration (`config/image.php`, env-driven) . . . 3

3 Quickstart **3**

4 Method reference **3**

4.1 `dimensions()` / `detectFormat()` / `validate()` . . . 3

4.2 `resize()` . . . 4

4.3 `convert()` . . . 4

4.4 `variants()` . . . 4

4.5 `orient()` . . . 5

4.6 `crop()` . . . 5

4.7 Filters — `blur()` / `sharpen()` / `pixelate()` / `grayscale()` . . . 5

4.8 `adjustColor()` . . . 5

4.9 `watermarkImage()` / `watermarkText()` . . . 5

4.10 `placeholder()` . . . 6

5 Security: what's enforced automatically **6**

5.1 Imagick-specific: real resource limits, not just policy.xml . . . 6

6 Performance notes **7**

7 Common pitfalls **7**

8 Quick reference **8**

1 What this guide is

`hkm-plugin-image` provides the `image.processing` domain — resize, convert, crop, filter, watermark and placeholder-hash generation over GD, Imagick or libvips, behind one published contract, `ImageProcessorContract`. It has no persistence of its own: every method is a pure byte-string transform, `string $contents` in, `string $contents` out. Pair it with `StoragePort` (e.g. `hkm-plugin-storage`) to keep a result.

Every number in this guide was recorded, not written

`docs/generators/probe_image.php` runs every method below against a real generated 1600x900 JPEG (167.3 KB) through whichever engines are actually installed on the machine that generated this PDF — here: GD, Imagick, Vips. Regenerate with `php probe_image.php && python3 build_image.py`. If an engine's output changes, the numbers in this document change with it instead of going stale.

2 Install and choose a driver

```
composer require alfacode-team/hkm-plugin-image
# or, from a project:
hkm plugins add image
```

A route or job that resolves `ImageProcessorContract` MUST declare `"requires": ["image.processing"]` in its `module.json` — this is an on-demand module, not an essential one; the contract is unbound otherwise.

2.1 Three engines, one contract

	gd (default)	imagick	vips
Dependency	none — ships with nearly every PHP build	ext-imagick	jcupitt/vips + ext-ffi + the libvips shared library
AVIF	only if GD was built against libavif	depends on the ImageMagick build	depends on the libvips build
EXIF auto-orient	hand-implemented rotate/flip math	native	native and automatic (built into resize/thumbnail)
`ResizeMode::Smart`	falls back to Cover	falls back to Cover	real content-aware crop
Animated source	first frame only	first frame only	first frame only

`IMAGE_DRIVER=auto` picks `vips` when installed, else `imagick` when loaded, else `gd`. An explicitly named driver that is not installed fails the boot immediately — `gd` is the default specifically because it is almost never missing.

2.2 Configuration (`config/image.php`, env-driven)

Key	Default	Meaning
IMAGE_DRIVER	gd	gd, imagick, vips, or auto
IMAGE_DEFAULT_FORMAT	webp	used by `variants()` when no format is passed
IMAGE_QUALITY	82	1–100, applied to jpeg/webp/avif; mapped to PNG's 0–9 scale
IMAGE_MAX_BYTES	10485760	upload size ceiling — enforced by every method now, not only `validate()`
IMAGE_ALLOWED_MIME	jpeg, png, webp, gif, avif	comma list — `validate()`'s per-route allowlist
IMAGE_MAX_WIDTH / _HEIGHT	8000	decompression-bomb ceiling — enforced structurally (see the Security section)
IMAGE_VARIANT_PRESETS	thumbnail:150, medium:300, large:1024	parsed by `image_variant_presets()`
IMAGE_WATERMARK_TTF_PATH	unset	GD-only: a `.ttf`/`.otf` path for `watermarkText()`

3 Quickstart

```
$image   = $container->make(ImageProcessorContract::class);
$storage = $container->make(StoragePort::class);

// 1. Validate BEFORE doing anything else with untrusted upload bytes — this
//    is still where a per-route MIME allowlist lives.
$image->validate(
    $bytes,
    maxBytes: (int) image_config('max_bytes'),
    allowedMimeTypes: image_allowed_mime_types(),
);

// 2. Transform.
$bytes = $image->watermarkText($bytes, '(c) 2026', Position::BottomRight, opacityPercent: 60);

// 3. Several named renditions in one pass.
foreach ($image->variants($bytes, image_variant_presets(), ImageFormat::Webp) as $name => $variant) {
    $storage->store($variant->contents, "{$name}.webp", "uploads/{$id}");
}

// 4. Ship a blur-up placeholder alongside the URL.
$placeholder = $image->placeholder($bytes);
```

4 Method reference

Every example below actually ran against the GD engine (the default driver) on a 1600x900 JPEG (167.3 KB). Where a number differs meaningfully by engine, both are shown.

4.1 `dimensions()` / `detectFormat()` / `validate()`

```
$image->dimensions($bytes);      // Dimensions { width, height } — header read only
$image->detectFormat($bytes);    // ImageFormat::Jpeg
$image->validate($bytes, maxBytes: 10_000_000, allowedMimeTypes: ['image/jpeg']);
```

Recorded

`validate($photo, 10_000_000, ['image/jpeg'], maxWidth: 100)` on the 1600px-wide source threw: `Image: width 1600px exceeds the 100px limit.`

4.2 `resize()`

```
$image->resize($bytes, width: 300, height: 200, mode: ResizeMode::Cover);
```

Mode	Output	Size	GD time
Cover	300x200	9.5 KB	46.786 ms
Contain	300x169	9.1 KB	40.714 ms
Stretch	300x200	10.1 KB	40.342 ms

Mode	Behavior	Upscales?
Cover (default)	fills the exact target box, center-crops the overflow	never
Contain	fits within the box, preserves aspect, no crop	never
Stretch	forces the exact target size, ignoring aspect	the only mode that does
Smart	like Cover, but crops over the most visually significant region — **Vips only**	never

4.3 `convert()`

```
$webp = $image->convert($bytes, ImageFormat::Webp);
```

Engine	WebP size	Ratio	Time
GD	42.0 KB	25.1% of source	166.833 ms
Imagick	42.0 KB	25.1% of source	146.56 ms
Vips	42.0 KB	25.1% of source	212.379 ms

4.4 `variants()`

```
$variants = $image->variants(
    $bytes,
    presets: ['thumbnail' => 150, 'medium' => 300, 'large' => 1024],
    format: ImageFormat::Webp,
);
// => ['thumbnail' => ImageVariant, 'medium' => ImageVariant, 'large' => ImageVariant]
```

Preset	Output	Size
thumbnail	150x84	2.8 KB
medium	300x169	5.8 KB
large	1024x576	22.0 KB

One decode, not one per preset

As of this version, `variants()` decodes the source ONCE and reuses it for every preset (GD resamples the same decoded image per preset; Imagick clones it per preset, since its resize methods mutate in place). Recorded total time for all 3 presets above: GD 227.919 ms, Imagick 257.451 ms, Vips 86.607 ms — well under 3x any single resize, because the expensive full decode no longer repeats per preset.

4.5 `orient()`

```
$upright = $image->orient($bytes);   // reads EXIF Orientation, rotates, resets the tag
```

Recorded

A hand-built 200x100 JPEG carrying a real EXIF Orientation=6 tag (needs a 90 deg correction) came back `100×200` from `200×100` — the width/height swap confirms the rotation actually ran, on every engine tested here.

4.6 `crop()`

```
$avatar = $image->crop($bytes, x: 100, y: 100, width: 500, height: 350);
```

Clamps to the source's actual bounds rather than erroring on an out-of-range rectangle — a caller-driven crop UI does not need to pre-validate its own math. Recorded: 500x350, 13.2 KB.

4.7 Filters — `blur()` / `sharpen()` / `pixelate()` / `grayscale()`

```
$image->blur($bytes, sigma: 3.0);
$image->sharpen($bytes);
$image->pixelate($bytes, blockSize: 12);   // privacy redaction — harder to reverse than blur, not cryptographically safe
$image->grayscale($bytes);
```

Filter	GD size	GD time
`blur()`	69.0 KB	643.295 ms
`sharpen()`	156.7 KB	206.212 ms
`pixelate()`	68.2 KB	43.871 ms
`grayscale()`	66.4 KB	39.629 ms

4.8 `adjustColor()`

```
$image->adjustColor($bytes, brightness: 10, contrast: 15, saturation: 20);
// each parameter is a -100..100 delta from "no change" at 0
```

Recorded output: 92.4 KB. GD has no native saturation filter — it falls back to a per-pixel blend loop, meaningfully slower than Imagick's/Vips' native saturation on anything bigger than a thumbnail (see the Performance notes section).

4.9 `watermarkImage()` / `watermarkText()`

```
$branded = $image->watermarkImage($bytes, $logoBytes, Position::BottomRight, opacityPercent: 70);
$dated   = $image->watermarkText($bytes, '(c) 2026 Acme', Position::Bottom, fontSize: 18, textColor: '#FFFFFF');
```

Recorded: image watermark 86.6 KB, text watermark 86.6 KB. Nine-point gravity (`Position::TopLeft` through `BottomRight`, plus `Center`), opacity and margins are identical across engines — the RENDERED TEXT is not: GD uses 5 built-in bitmap fonts unless `IMAGE_WATERMARK_TTF_PATH` is set, Imagick/

Vips need a font resolvable through the host's font system.

4.10 `placeholder()`

```
$hash = $image->placeholder($bytes);   // base64 ThumbHash — ~25–38 bytes decoded
```

Recorded

A real base64 ThumbHash for the 1600x900 source, GD engine: `HxgCbJCVjTZn/FcaGmeIgJkIeQ==` (28 base64 chars). Decode client-side into an instant blurred preview shown before the real image loads — or use `ThumbHash::averageColor()` for a CSS `background-color` fallback that needs no client-side decoder at all.

5 Security: what's enforced automatically

`validate()` is still the right place for a per-route MIME allowlist and an explicit dimension ceiling narrower than the engine's own default. But every engine's `decode()` now ALSO enforces `IMAGE_MAX_BYTES`/`IMAGE_MAX_WIDTH`/`IMAGE_MAX_HEIGHT` structurally — so skipping `validate()` no longer opens an unbounded decompression-bomb path through `resize()`, `convert()`, or any other method.

Recorded — the guard firing with no validate() call at all

Calling `convert()` directly on the 1600x900 source against an engine configured with `maxWidth: 400, maxHeight: 400` threw: `Image: width 1600px exceeds the 400px limit.` — on every engine tested here (GD, Imagick, Vips).

5.1 Imagick-specific: real resource limits, not just policy.xml

`ImagickImageEngine` additionally installs real `Imagick::setResourceLimit()` bounds once per construction — a second, independent backstop, since ImageMagick is the most exploit-prone of the three engines via non-raster coders/delegates, and its behavior otherwise depends entirely on the host's own, often-unconfigured, `policy.xml`.

Resource	Value at default config (8000x8000 / 10 MB)
WIDTH / HEIGHT	8000 / 8000 px
AREA	64,000,000 px
MEMORY / MAP	976 MB
DISK	2048 MB
TIME	30 s

Process-global — by design, verified during development

`Imagick::setResourceLimit()` mutates ImageMagick's OWN process-wide limiter even when called on an instance — a limit set via one `Imagick` object rejects a decode on a completely separate one with no limits of its own. Safe here because every value comes from this plugin's own config, re-applied identically every request.

6 Performance notes

These are single-run, single-machine timings recorded on the machine that built this PDF — useful for relative comparison between engines and operations, not a rigorous benchmark. Do not treat the millisecond columns above as production guarantees.

Engine	convert()	resize() Cover	variants() (3 presets)
GD	166.833 ms	46.786 ms	227.919 ms
Imagick	146.56 ms	79.726 ms	257.451 ms
Vips	212.379 ms	10.043 ms	86.607 ms

`vips` uses shrink-on-load (`thumbnail_buffer()`) for `resize()`/`variants()` — the decoder can skip full-resolution decoding once it knows the target size — which is why it tends to lead on exactly those two operations. `gd` and `imagick` decode at full resolution first; both now decode the source ONCE per `variants()` call rather than once per preset (see the `variants()` entry in the Method reference above).

`placeholder()` on Imagick uses a `jpeg:size` shrink-on-load decode hint (verified: a 2000x2000 JPEG decoded at 250x250, not full-size, with this hint set) — large source images no longer have to be decoded at full resolution just to build a ~30-byte hash. `gd` has no such hint available and decodes fully; `vips`'s `placeholder()` was already using `thumbnail_buffer()`.

GD's `adjustColor()` saturation and `watermarkImage()`/`watermarkText()` opacity scaling are both O(width x height) pure-PHP pixel loops (GD has no native saturation filter, and `imagecopymerge()` silently flattens per-pixel alpha, so both need hand-rolled per-pixel math). Meaningfully slower than Imagick's/Vips' native equivalents on anything bigger than a thumbnail — prefer `IMAGE_DRIVER=imagick` or `vips` if your workload does either routinely on full-size images.

7 Common pitfalls

Don't	Because
`store()`/`convert()` a multi-hundred-MB image synchronously in a request handler	no streaming variant here (unlike `StoragePort`) — route large batch work through a Worker job
Assume `resize()`/`convert()` preserve EXIF/IPTC/XMP	every engine strips metadata on every call, deliberately — call `orient()` FIRST if you need the rotation
Resolve `ImageProcessorContract` from a route that didn't declare `"requires": ["image.processing"]`	the contract is not bound otherwise — surfaces as an unresolvable binding at request time
Treat `pixelate()` as reversible-enough redaction for a genuinely sensitive region at a small block size	harder to reverse than a Gaussian blur, not cryptographically safe — crop the region out entirely when in doubt
Assume `watermarkText()` looks the same across engines	GD's built-in bitmap fonts vs. Imagick's/Vips' system-font rendering are not visually interchangeable
Rely on `ResizeMode::Smart` outside `vips`	GD/Imagick silently fall back to plain Cover — documented, not a bug, but not "smart" either

8 Quick reference

Method	Signature (trimmed)
dimensions	`dimensions(string $contents): Dimensions`
detectFormat	`detectFormat(string $contents): ImageFormat`
validate	`validate($contents, int $maxBytes, array $allowedMimeTypes, ?int $maxWidth = null, ?int $maxHeight = null): void`
resize	`resize($contents, int $width, ?int $height = null, ResizeMode $mode = Cover): string`
convert	`convert($contents, ImageFormat $format, ?int $quality = null): string`
stripMetadata	`stripMetadata($contents): string`
orient	`orient($contents): string`
variants	`variants($contents, array $presets, ?ImageFormat $format = null, ?int $quality = null, ResizeMode $mode = Cover): array`
crop	`crop($contents, int $x, int $y, int $width, int $height): string`
blur	`blur($contents, float $sigma = 2.0): string`
sharpen	`sharpen($contents): string`
pixelate	`pixelate($contents, int $blockSize = 8): string`
grayscale	`grayscale($contents): string`
adjustColor	`adjustColor($contents, float $brightness = 0, float $contrast = 0, float $saturation = 0): string`
watermarkImage	`watermarkImage($contents, $overlayContents, Position $position = BottomRight, int $opacityPercent = 100, int $marginX = 10, int $marginY = 10): string`
watermarkText	`watermarkText($contents, string $text, Position $position = BottomRight, int $fontSize = 16, string $textColor = '#FFFFFF', int $opacityPercent = 100, int $marginX = 10, int $marginY = 10): string`
placeholder	`placeholder($contents): string`